Table of Contents

..... 1
Task 1: 5-Band Equalizer Design 1
Frequency Response Analysis 2
Equalizer Processing Function 4
Task 2: Audio Presets 5
Task 3: Process signals 7
Task 4: Bird Vocalization Enhancement 8
ORIGINAL VS PROCESSED SIDE-BY-SIDE 11
i. & iii. Best spectrogram parameters 12
ii. Count distinct vocalizations 13
Show isolation 15


```
clear all; close all; clc
```

Task 1: 5-Band Equalizer Design

```
fs = 44100; dt = 1/fs;
center_freqs = [60, 250, 1000, 4000, 16000];
Q = 1.5; bandwidths = center_freqs / Q;
R = 1000;

% Initialize filters struct
filters = struct('center', cell(1,5), 'f_low', cell(1,5), 'f_high',
cell(1,5), ...
    'alpha_hp', cell(1,5), 'alpha_lp', cell(1,5), 'C_hp', cell(1,5), 'C_lp',
cell(1,5));

fprintf('5-BAND EQUALIZER DESIGN \nBand\tCenter\tBW\tRange\n');
for i = 1:5
    f_low = max(20, center_freqs(i) - bandwidths(i)/2);
    f_high = min(20000, center_freqs(i) + bandwidths(i)/2);
    fprintf('Band %d\t%.0f\t%.0f\t%.0f-%.0f\n', i, center_freqs(i),
bandwidths(i), f_low, f_high);

    % Calculate filter parameters
    C_hp = 1/(2*pi*f_low*R); tau_hp = R*C_hp; alpha_hp = dt/tau_hp;
    C_lp = 1/(2*pi*f_high*R); tau_lp = R*C_lp; alpha_lp = dt/tau_lp;

    filters(i).center = center_freqs(i);
    filters(i).f_low = f_low;
    filters(i).f_high = f_high;
    filters(i).alpha_hp = alpha_hp;
    filters(i).alpha_lp = alpha_lp;
    filters(i).C_hp = C_hp;
    filters(i).C_lp = C_lp;
end
```

```
5-BAND EQUALIZER DESIGN
Band    Center    BW    Range
```

```
Band 1     60     40     40-80
Band 2     250     167     167-333
Band 3     1000     667     667-1333
Band 4     4000     2667     2667-5333
Band 5     16000     10667     10667-20000
```

Frequency Response Analysis

```
f_axis = logspace(log10(20), log10(20000), 1000);
H_bands = zeros(5, length(f_axis));
figure('Position', [100, 100, 1200, 500]);

for i = 1:5
    f_low = filters(i).f_low; f_high = filters(i).f_high;
    tau_hp = R*filters(i).C_hp; tau_lp = R*filters(i).C_lp;

    s = 1j*2*pi*f_axis;
    H_hp = s./(s + 1/tau_hp);
    H_lp = (1/tau_lp)./(s + 1/tau_lp);
    H_bands(i,:) = H_hp .* H_lp;

    subplot(2,3,i);
    semilogx(f_axis, 20*log10(abs(H_bands(i,:)) + eps), 'b', 'LineWidth',
2); hold on;

    % Mark -3dB points and center
    [~, idx_low] = min(abs(f_axis - f_low));
    [~, idx_high] = min(abs(f_axis - f_high));
    [~, idx_center] = min(abs(f_axis - center_freqs(i)));

    plot(f_axis(idx_low), 20*log10(abs(H_bands(i,idx_low))), 'ro',
'MarkerSize', 8);
    plot(f_axis(idx_high), 20*log10(abs(H_bands(i,idx_high))), 'ro',
'MarkerSize', 8);
    plot(center_freqs(i), 20*log10(abs(H_bands(i,idx_center))), 'gs',
'MarkerSize', 8);

    xlabel('Hz'); ylabel('dB'); title(sprintf('Band %d: %.0f Hz', i,
center_freqs(i)));
    grid on; xlim([20,20000]); ylim([-40,5]);
    legend('Response','-3dB','Center','Location','southwest');
end

% Architecture diagram
subplot(2,3,6); axis off;
text(0.1,0.9,'EQUALIZER ARCHITECTURE:','FontWeight','bold');
text(0.1,0.8,'Input x[n] →');
for i = 1:5
    text(0.15,0.8-i*0.1, sprintf('Band %d (%.0f Hz) → G%d →', i,
center_freqs(i), i));
end
text(0.1,0.2,'Sum → Output y[n]');
text(0.1,0.1,'Gain range: -15 dB to +15 dB per band');
```

```
f_axis = logspace(log10(20), log10(20000), 1000);
figure('Position', [100, 100, 1400, 800]);
for i = 1:5
    f_low = filters(i).f_low;
    f_high = filters(i).f_high;
    % Create continuous-time transfer function
    % Bandpass = HPF * LPF
    % HPF: s/(s+ω_low) where ω_low = 2πf_low
    % LPF: ω_high/(s+ω_high) where ω_high = 2πf_high
    % Using s-domain representation
    w_low = 2*pi*f_low;
    w_high = 2*pi*f_high;
    % Transfer function: H(s) = [s/(s+w_low)] * [w_high/(s+w_high)]
    % Multiply out: H(s) = (s*w_high) / (s^2 + (w_low+w_high)s +
w_low*w_high)
    b = [w_high, 0];  % Numerator: s*w_high
    a = [1, w_low+w_high, w_low*w_high];  % Denominator
    % Frequency response using freqs (continuous-time)
    [H, w] = freqs(b, a, 2*pi*f_axis);
    mag = abs(H);
    phase = angle(H) * 180/pi;

    % Plot magnitude
    subplot(5, 2, 2*i-1);
    semilogx(f_axis, 20*log10(mag + eps), 'b', 'LineWidth', 2);
    title(sprintf('Band %d (%.0f Hz) - Magnitude', i, center_freqs(i)));
    xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
    grid on; xlim([20, 20000]); ylim([-40, 5]);

    % Plot phase
    subplot(5, 2, 2*i);
    semilogx(f_axis, phase, 'r', 'LineWidth', 2);
    title(sprintf('Band %d (%.0f Hz) - Phase', i, center_freqs(i)));
    xlabel('Frequency (Hz)'); ylabel('Phase (degrees)');
    grid on; xlim([20, 20000]);
end

% Figure 3: Impulse responses using lsim
figure('Position', [100, 100, 1200, 600]);

for i = 1:5
    % Get filter parameters
    f_low = filters(i).f_low;
    f_high = filters(i).f_high;
    w_low = 2*pi*f_low;
    w_high = 2*pi*f_high;
    b = [w_high, 0];
    a = [1, w_low+w_high, w_low*w_high];

    % Impulse response using lsim
    t = 0:1/fs:0.01;  % 10ms impulse response
    impulse = [1; zeros(length(t)-1, 1)];
    [h, t_out] = lsim(tf(b, a), impulse, t);
```

```
    subplot(2,3,i);
    plot(t_out*1000, h, 'LineWidth', 2);
    title(sprintf('Band %d (%.0f Hz) Impulse Response', i, center_freqs(i)));
    xlabel('Time (ms)'); ylabel('Amplitude');
    grid on;
    xlim([0, 10]);
end
```

Equalizer Processing Function

```
function y = process_eq(x, filters, gains_db, fs)
    % Process input through 5-band equalizer
    % x: input signal
    % filters: filter parameters struct
    % gains_db: 5-element vector of gains in dB
    % fs: sampling rate

    dt = 1/fs;
    y = zeros(size(x));

    for b = 1:5
        % Convert dB gain to linear
        g = 10^(gains_db(b)/20);

        % Get filter coefficients (adaptive to fs)
        alpha_hp = min(dt / (filters(b).C_hp * 1000), 0.99);
        alpha_lp = min(dt / (filters(b).C_lp * 1000), 0.99);

       % High-pass filter
        y_hp = zeros(size(x));
        y_hp(1) = x(1);

        for n = 1:length(x)-1
            y_hp(n+1) = (1 - alpha_hp)*y_hp(n) + alpha_hp*(x(n+1) - x(n));
        end

        % Low-pass filter
        y_band = zeros(size(x));
        y_band(1) = y_hp(1);
        for n = 1:length(x)-1
            y_band(n+1) = (1 - alpha_lp)*y_band(n) + alpha_lp*y_hp(n);
        end

        % Apply gain and add to output
        y = y + g * y_band;
    end

    % Prevent clipping
    if max(abs(y)) > 1
        y = 0.95 * y / max(abs(y));
    end
end
```

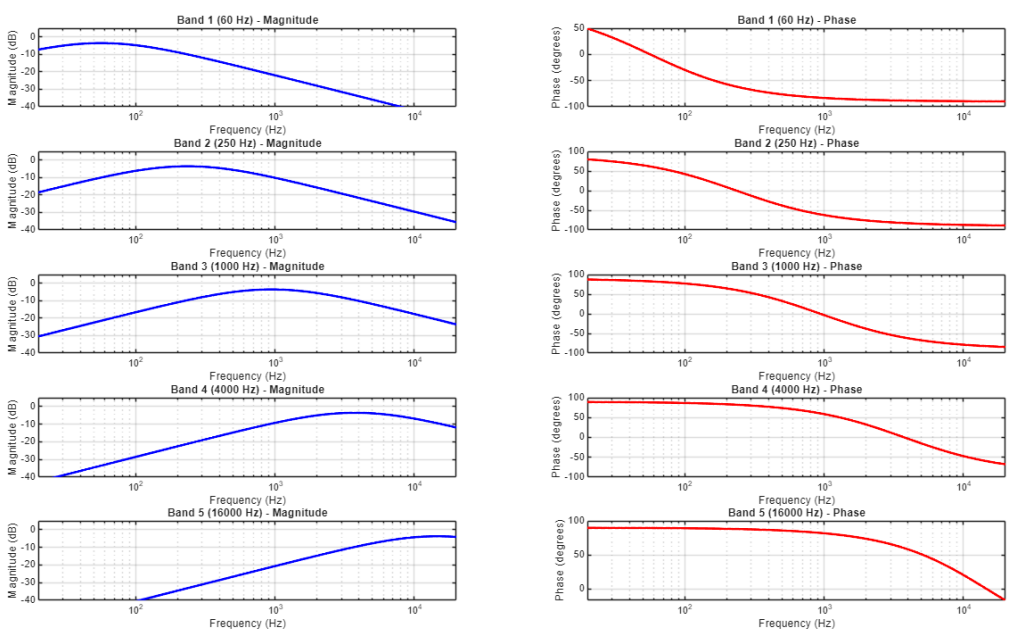
Processed audio files saved

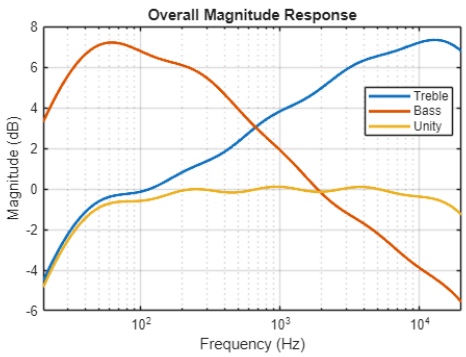
Task 2: Audio Presets

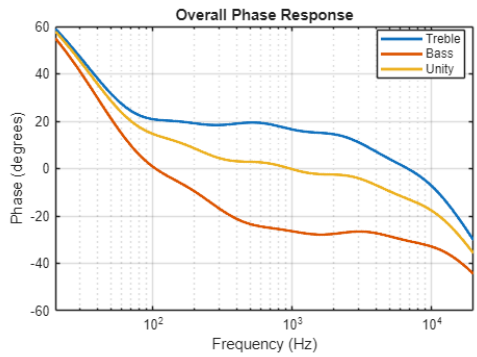
```
% Gains in dB for each band: [60Hz, 250Hz, 1kHz, 4kHz, 16kHz]
% Linear: gain = 10^dB/20, gain_linear = db2mag(gain_dB)

% Treble Boost: [0 0 3 6 9];
preset.treble = [0 0 1.2589 1.9953 2.8184];

% Bass Boost: [9 6 0 -3 -6];
preset.bass = [2.8184 1.9953 0 0.7079 0.5012];

% Unity: [0 0 0 0 0];
preset.unity = [1 1 1 1 1];

fprintf('\n Preset Gain Settings (dB)\n');
fprintf('Treble Boost: [%d %d %d %d %d]\n', preset.treble);
fprintf('Bass Boost:   [%d %d %d %d %d]\n', preset.bass);
fprintf('Unity:        [%d %d %d %d %d]\n', preset.unity);

presets_db = {
    'Treble Boost', [0, 0, 3, 6, 9];
    'Bass Boost',   [9, 6, 0, -3, -6];
    'Unity',        [0, 0, 0, 0, 0]
};

f_axis = logspace(log10(20), log10(20000), 1000);
figure('Position', [100, 100, 1200, 800]);

for p = 1:3
    preset_name = presets_db{p,1};
    gains_db = presets_db{p,2};
    gains_linear = 10.^(gains_db/20);

    % Calculate total frequency response
    H_total = zeros(size(f_axis));

    for b = 1:5
        % Get band transfer function
        f_low = filters(b).f_low;
        f_high = filters(b).f_high;
        w_low = 2*pi*f_low;
        w_high = 2*pi*f_high;

        % Bandpass = HPF * LPF
        b_coeff = [w_high, 0];
        a_coeff = [1, w_low+w_high, w_low*w_high];

        % Get frequency response
        [H_band, ~] = freqs(b_coeff, a_coeff, 2*pi*f_axis);
```

```
        % Add weighted contribution
        H_total = H_total + gains_linear(b) * H_band;
    end

    % Plot magnitude response
    subplot(2,2,1);
    semilogx(f_axis, 20*log10(abs(H_total) + eps), 'LineWidth', 2);
    hold on;
    title('Overall Magnitude Response');
    xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
    grid on; xlim([20, 20000]);

    % Plot phase response
    subplot(2,2,2);
    semilogx(f_axis, unwrap(angle(H_total))*180/pi, 'LineWidth', 2);
    hold on;
    title('Overall Phase Response');
    xlabel('Frequency (Hz)'); ylabel('Phase (degrees)');
    grid on; xlim([20, 20000]);
end

subplot(2,2,1); legend('Treble', 'Bass', 'Unity', 'Location', 'best');
subplot(2,2,2); legend('Treble', 'Bass', 'Unity', 'Location', 'best');
```

```
 Preset Gain Settings (dB)
Treble Boost: [0 0 1.258900e+00 1.995300e+00 2.818400e+00]
Bass Boost:   [2.818400e+00 1.995300e+00 0 7.079000e-01 5.012000e-01]
Unity:        [1 1 1 1 1]
```







Task 3: Process signals

Load Giant Steps audio

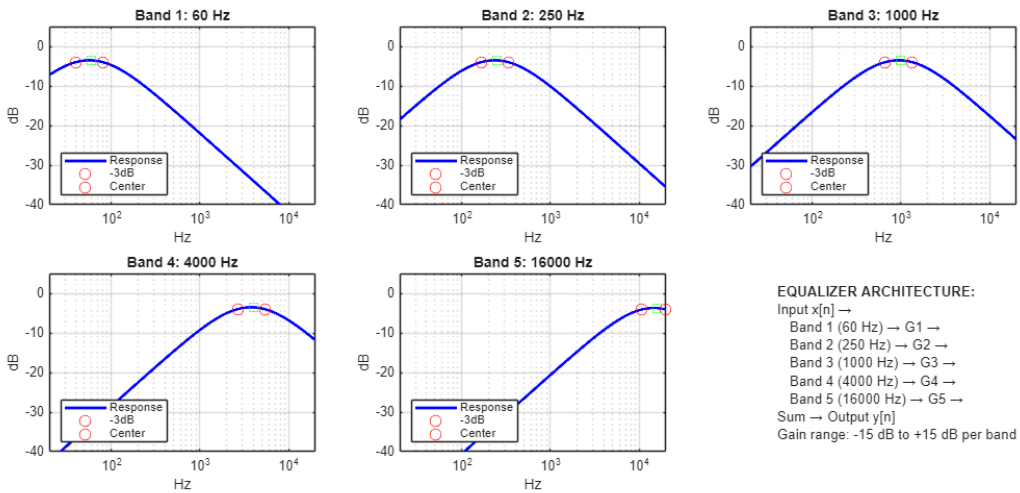
```
[x_giant, ~] = audioread('Giant Steps Bass Cut.wav');

% Load Space Station audio
[x_space, fs] = audioread('Space Station - Treble Cut.wav');

% Convert stereo to mono if needed
if size(x_giant,2) > 1
    x_giant = mean(x_giant,2);
end

if size(x_space,2) > 1
    x_space = mean(x_space,2);
end

% Apply equalizer presets to Giant Steps
y_giant_treble = process_eq(x_giant, filters, preset.treble, fs);
y_giant_bass = process_eq(x_giant, filters, preset.bass, fs);
y_giant_unity = process_eq(x_giant, filters, preset.unity, fs);


% Apply equalizer presets to Space Station
y_space_treble = process_eq(x_space, filters, preset.treble, fs);
```

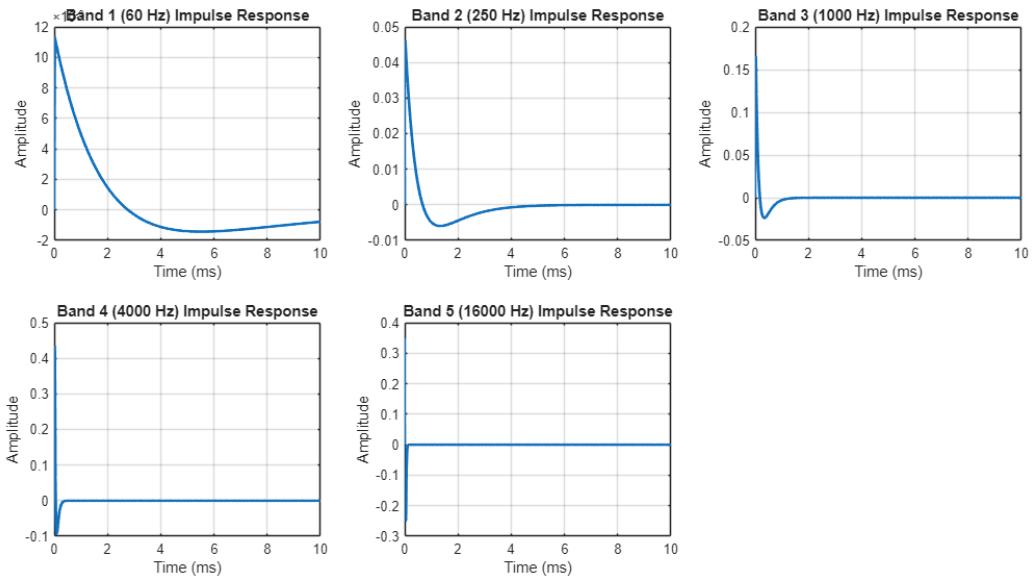
```
y_space_bass = process_eq(x_space, filters, preset.bass, fs);
y_space_unity = process_eq(x_space, filters, preset.unity, fs);

% Save audio
audiowrite('giant_treble.wav', y_giant_treble, fs);
audiowrite('giant_bass.wav', y_giant_bass, fs);
audiowrite('giant_unity.wav', y_giant_unity, fs);

audiowrite('space_treble.wav', y_space_treble, fs);
audiowrite('space_bass.wav', y_space_bass, fs);
audiowrite('space_unity.wav', y_space_unity, fs);

fprintf('\nProcessed audio files saved\n');
```

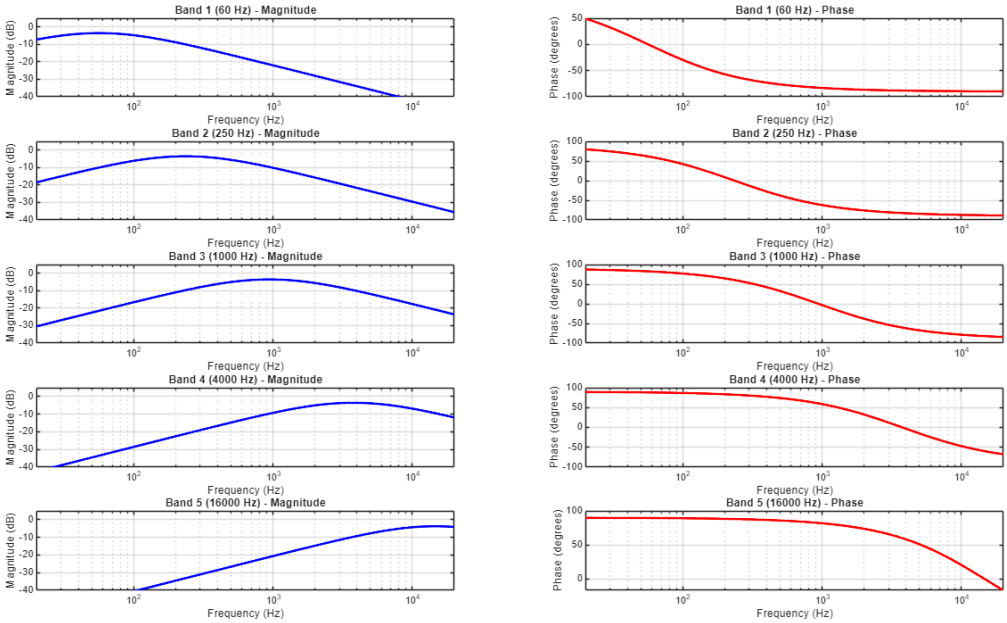
Task 4: Bird Vocalization Enhancement

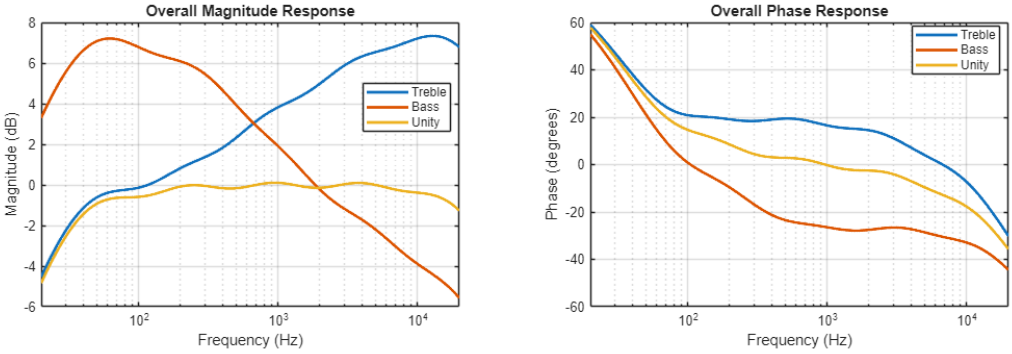
```
[x_bird, fs_bird] = audioread('SNR Recording 2026-02-15 08_58.wav');
if size(x_bird,2) > 1
    x_bird = mean(x_bird,2);
end

wind_footstep_leaf = [-25, -25, -15, -6, 0];


y_clean = process_eq(x_bird, filters, wind_footstep_leaf, fs_bird);
y_clean_boosted = y_clean * 3;

fprintf('PLAYING ORIGINAL (with wind, footsteps, leaf rustle)');
 sound(x_bird, fs_bird);


 pause(length(x_bird)/fs_bird + 1);

% Play cleaned version
fprintf('PLAYING CLEANED VERSION (noise reduced)');
 sound(y_clean_boosted, fs_bird);
```

PLAYING ORIGINAL (with wind, footsteps, leaf rustle)PLAYING CLEANED VERSION (noise reduced)


Band 1: 60 Hz
Band 2: 250 Hz
Band 3: 1000 Hz
Band 4: 4000 Hz
Band 5: 16000 Hz
dB
Hz
Response
-3dB
Center
EQUALIZER ARCHITECTURE:
Input x[n] →
Band 1 (60 Hz) → G1 →
Band 2 (250 Hz) → G2 →
Band 3 (1000 Hz) → G3 →
Band 4 (4000 Hz) → G4 →
Band 5 (16000 Hz) → G5 →
Sum → Output y[n]
Gain range: -15 dB to +15 dB per band

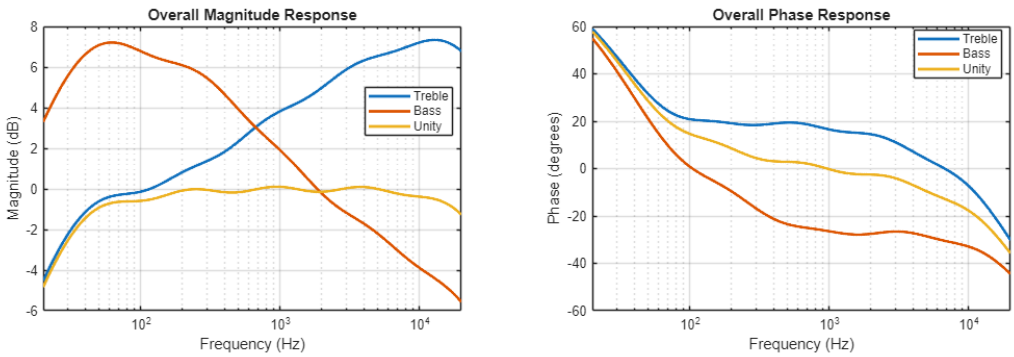

Band 1 (60 Hz) Impulse Response
Band 2 (250 Hz) Impulse Response
Band 3 (1000 Hz) Impulse Response
Band 4 (4000 Hz) Impulse Response
Band 5 (16000 Hz) Impulse Response
Amplitude
Time (ms)

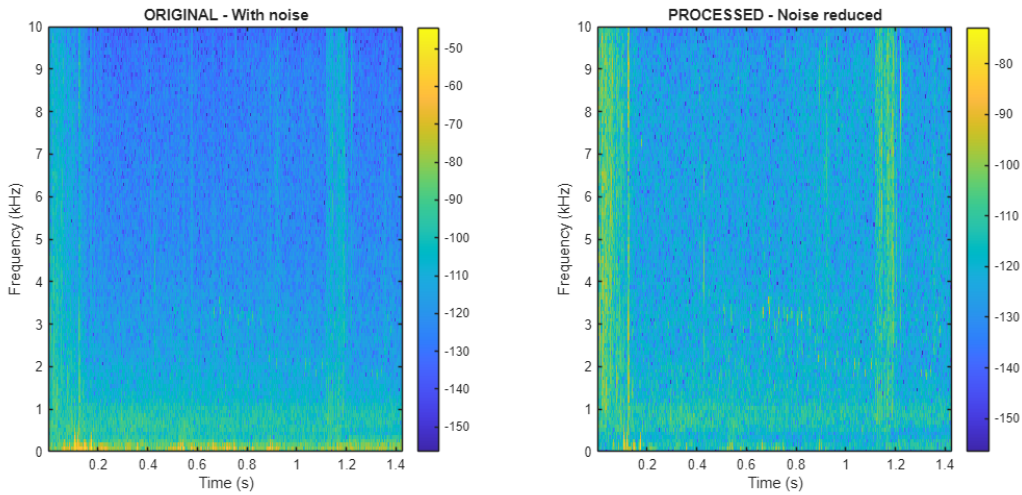

Band 1 (60 Hz) - Magnitude
Band 1 (60 Hz) - Phase
Band 2 (250 Hz) - Magnitude
Band 2 (250 Hz) - Phase
Band 3 (1000 Hz) - Magnitude
Band 3 (1000 Hz) - Phase
Band 4 (4000 Hz) - Magnitude
Band 4 (4000 Hz) - Phase
Band 5 (16000 Hz) - Magnitude
Band 5 (16000 Hz) - Phase
Magnitude (dB)
Phase (degrees)
Frequency (Hz)


Overall Magnitude Response
Overall Phase Response
Treble
Bass
Unity
Magnitude (dB)
Phase (degrees)
Frequency (Hz)

ORIGINAL VS PROCESSED SIDE-BY-SIDE

```
figure('Position', [100, 100, 1200, 500]);

% Left: Original
subplot(1,2,1);
spectrogram(x_bird, 512, 256, 512, fs_bird, 'yaxis');
title('ORIGINAL - With noise');
ylim([0, 10]); colorbar;
xlabel('Time (s)'); ylabel('Frequency (kHz)');

% Right: Processed
subplot(1,2,2);
spectrogram(y_clean, 512, 256, 512, fs_bird, 'yaxis');
title('PROCESSED - Noise reduced');
ylim([0, 10]); colorbar;
xlabel('Time (s)'); ylabel('Frequency (kHz)');
```

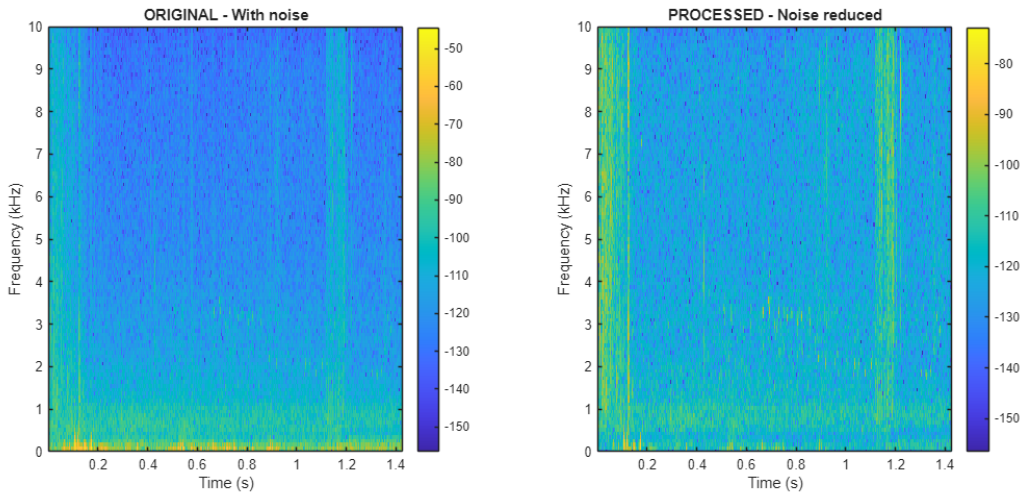


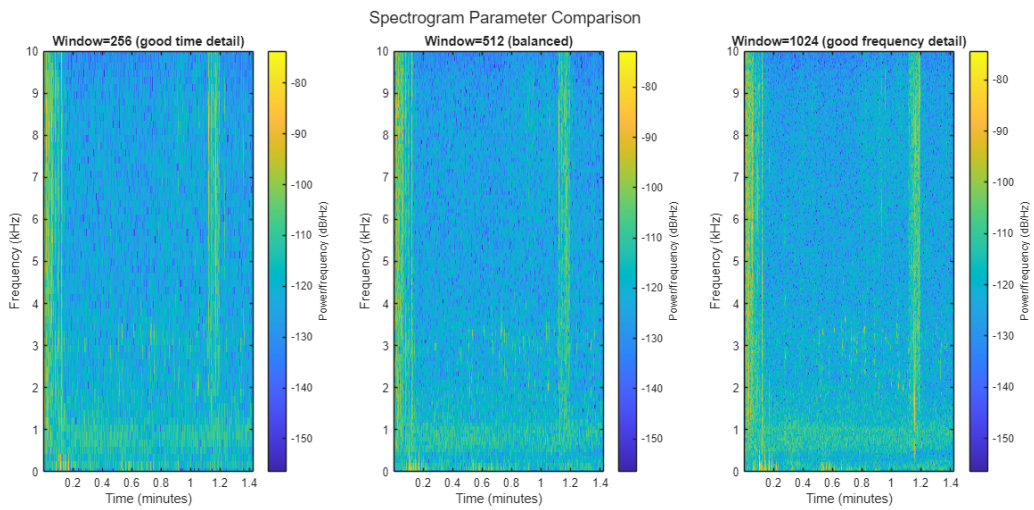


i. & iii. Best spectrogram parameters

```
figure('Position', [100, 100, 1400, 600]);

% Try different window sizes
subplot(1,3,1);
spectrogram(y_clean, 256, 128, 256, fs_bird, 'yaxis');
title('Window=256 (good time detail)'); ylim([0,10]);

subplot(1,3,2);
spectrogram(y_clean, 512, 256, 512, fs_bird, 'yaxis');
title('Window=512 (balanced)'); ylim([0,10]);

subplot(1,3,3);
spectrogram(y_clean, 1024, 512, 1024, fs_bird, 'yaxis');
title('Window=1024 (good frequency detail)'); ylim([0,10]);
sgtitle('Spectrogram Parameter Comparison');

fprintf('\nBest parameters: Window=512, Overlap=256, Freq range 0-10kHz\n');
```

```
Best parameters: Window=512, Overlap=256, Freq range 0-10kHz
```



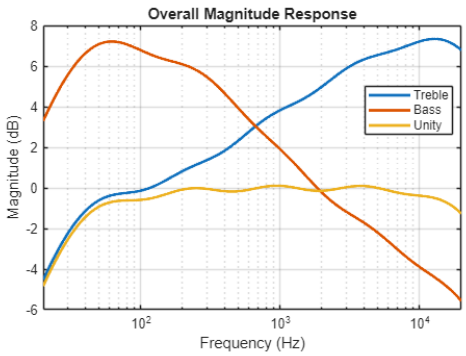


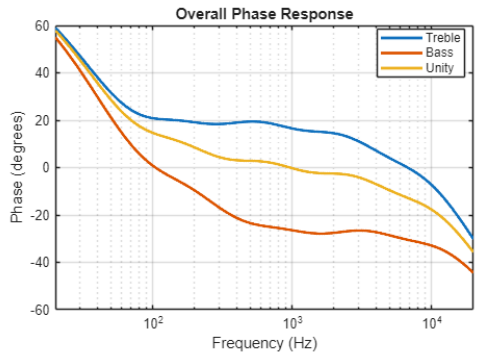
ii. Count distinct vocalizations

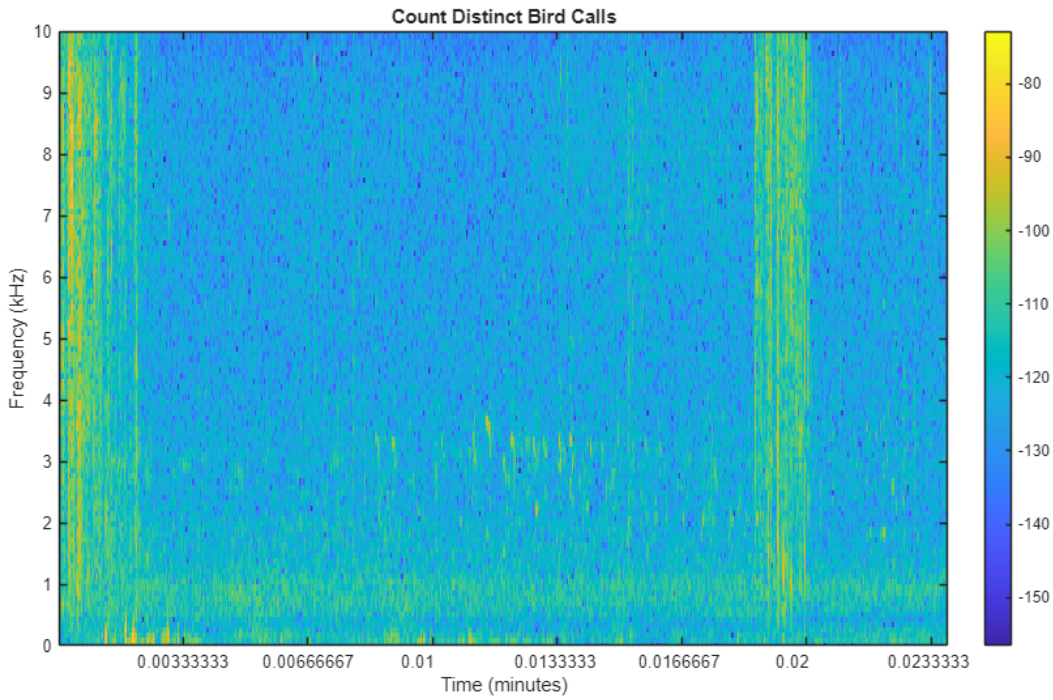
```
figure;
spectrogram(y_clean, 512, 256, 512, fs_bird, 'yaxis');
title('Count Distinct Bird Calls'); ylim([0,10]); colorbar;
xt = get(gca, 'XTick'); set(gca, 'XTickLabel', xt/60);
xlabel('Time (minutes)');
fprintf('\nIsolating specific birds:\n');

% Blue Jay (boost 4kHz)
bluejay_eq = [-20, -18, -12, +6, +3];
y_bluejay = process_eq(x_bird, filters, bluejay_eq, fs_bird);
fprintf('- Blue Jay: Boost 4kHz\n');

% Carolina Wren (boost highs)
wren_eq = [-20, -18, -6, +9, +6];
y_wren = process_eq(x_bird, filters, wren_eq, fs_bird);
fprintf('- Carolina Wren: Boost 4-16kHz\n');
```

```
% Northern Cardinal (boost mids)
cardinal_eq = [-20, -15, +3, +6, 0];
y_cardinal = process_eq(x_bird, filters, cardinal_eq, fs_bird);
fprintf('- Northern Cardinal: Boost 1-4kHz\n');
```

Isolating specific birds:
- Blue Jay: Boost 4kHz
- Carolina Wren: Boost 4-16kHz
- Northern Cardinal: Boost 1-4kHz

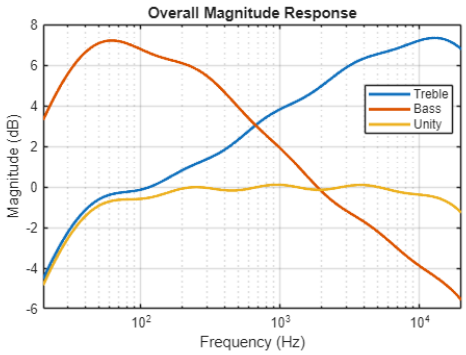


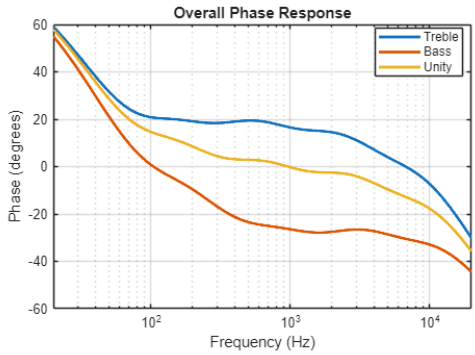


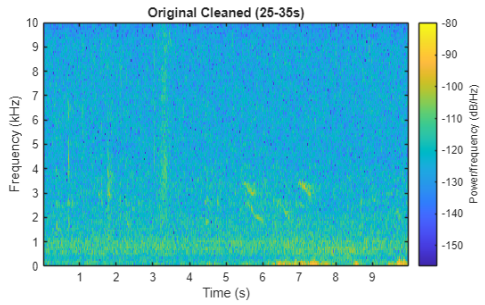


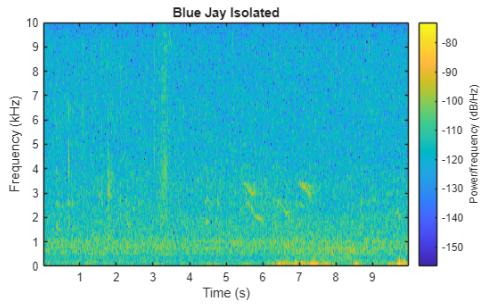
Show isolation

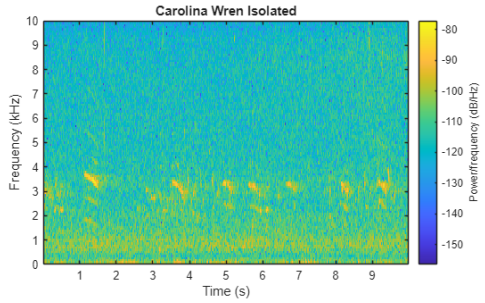
```
figure('Position', [100, 100, 1400, 800]);

subplot(2,2,1);
spectrogram(y_clean(25*fs_bird:35*fs_bird), 512, 256, 512, fs_bird, 'yaxis');
title('Original Cleaned (25-35s)'); ylim([0,10]);

subplot(2,2,2);
spectrogram(y_bluejay(25*fs_bird:35*fs_bird), 512, 256, 512, fs_bird,
'yaxis');
title('Blue Jay Isolated'); ylim([0,10]);

subplot(2,2,3);
spectrogram(y_wren(40*fs_bird:50*fs_bird), 512, 256, 512, fs_bird, 'yaxis');
title('Carolina Wren Isolated'); ylim([0,10]);

subplot(2,2,4);
spectrogram(y_cardinal(15*fs_bird:25*fs_bird), 512, 256, 512, fs_bird,
'yaxis');
title('Northern Cardinal Isolated'); ylim([0,10]);
```

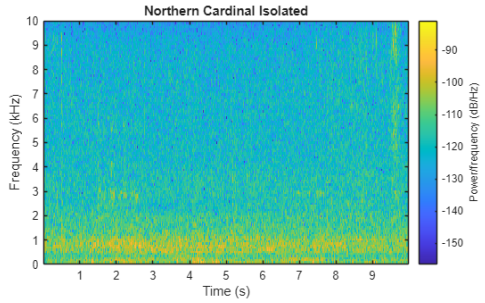












Published with MATLAB® R2025b